

DIZAJN PATERNI PONAŠANJA

Sistem HealthZone – poliklinika

Uvod

U prethodnim fazama razvoja sistema **HealthZone** definisani su strukturalni i kreacijski dizajn paterni koji su omogućili stabilnu osnovu za upravljanje korisnicima, terminima, nalazima i opremom. Međutim, kako sistem raste i obuhvata sve raznovrsnije scenarije korištenja – od različitih uloga (pacijent, doktor, medicinska sestra, administrator), preko evidencije termina i nalaza, pa do notifikacija i statistika – javlja se potreba za uvođenjem **paterna ponašanja**. Njihova primarna uloga je da opišu načine na koje objekti u sistemu međusobno komuniciraju i kako se raspodjeljuje odgovornost između njih, pri čemu se zadržava niska sprega i visoka prilagodljivost koda.

U nastavku su opisani konkretni paterni ponašanja koji se uvode u sistem HealthZone, njihova svrha, način implementacije i konkretna uloga u kontekstu poslovne logike poliklinike.

Strategy

Strategy patern omogućava definisanje familije algoritama i njihovu zamjenu u toku izvršavanja, pri čemu klijent koji koristi algoritam ne mora poznavati njegovu konkretnu implementaciju. U sistemu **HealthZone** različite uloge korisnika imaju različite poglede na iste podatke – doktor želi statistiku po pacijentima i specijalizaciji, administrator po cijeloj poliklinici, a pacijent po sopstvenoj historiji nalaza i termina.

To možemo implementirati:

- definiranjem interfejsa `StrategijaStatistike` s metodom `prikaziStatistiku()`;
- kreiranjem konkretnih implementacija: `StatistikaDoktor`, `StatistikaAdmin`, `StatistikaPacijent`;
- dodavanjem atributa `strategija` : `StrategijaStatistike` unutar klase `Korisnik`, koja se postavlja u zavisnosti od uloge prilikom prijave.

Korištenje:

Unutar sistema HealthZone Strategy patern se koristi za prikaz statistika i izvještaja. Doktor dobija pregled broja obavljenih termina, prosječne ocjene pacijenata i broja podnesenih zahtjeva za opremu. Administrator dobija agregirane podatke za cijelu polikliniku, dok pacijent vidi svoju historiju nalaza, recenzija i nadolazećih termina. Promjenom strategije bez izmjene postojećeg koda kontrolera moguće je jednostavno dodati nove tipove izvještaja (npr. statistiku po specijalizaciji ili po laboratorijskim nalazima).

State

State pattern omogućava da objekat mijenja svoje ponašanje u zavisnosti od svog internog stanja, tako da se čini kao da je promijenio klasu. U sistemu HealthZone najprirodniju primjenu nalazi u Životnom ciklusu **Termina**, koji prolazi kroz nekoliko jasno definisanih stanja: *aktivan*, *naCekanju*, *potvrden*, *otkazan* i *zavrsen*.

To možemo implementirati:

- definiranjem interfejsa StanjeTermina sa metodama potvrdi(), otkazi(), zavrsi();
- kreiranjem konkretnih klasa: AktivanTermin, NaCekanjuTermin, PotvrdenTermin, OtkazanTermin, ZavrsenTermin;
- dodavanjem atributa stanje : StanjeTermina u klasu Termin, koje se mijenja pozivom metoda iz odgovarajuće klase stanja.

Korištenje:

Termin u HealthZone sistemu reaguje različito na akcije korisnika u zavisnosti od trenutnog stanja. Na primjer, termin koji je u stanju Otkazan ne može biti ponovo potvrđen niti se nad njim može kreirati nalaz, dok termin u stanju Potvrden može preći u stanje Zavrsen tek nakon što doktor unese nalaz. Ovakvim pristupom uklanjaju se brojne if/else provjere iz kontrolera, a logika prelaza između stanja je jasno enkapsulirana u odgovarajućim klasama.

Template Method

Template Method pattern definiše kostur algoritma u apstraktnoj klasi, dok konkretne korake delegira podklasama. U sistemu HealthZone većina izvještaja prolazi kroz iste opšte korake – učitavanje podataka pacijenta, dodavanje nalaza, dodavanje terapija, formatiranje i štampanje – ali pojedinačni koraci se razlikuju u zavisnosti od vrste izvještaja.

To možemo implementirati:

- kreiranjem apstraktne klase IzvjestajGenerator s template metodom generisIzvjestaj();
- definiranjem apstraktnih koraka: dodajPodatkePacijenta(), dodajNalaze(), dodajTerapije(), dodajZakljucak();
- kreiranjem konkretnih implementacija: IzvjestajZaPacijenta, IzvjestajZaDoktora, IzvjestajZaLaboratoriju.

Korištenje:

U HealthZone sistemu Template Method pattern se primjenjuje pri generisanju medicinskih izvještaja. Izvještaj za pacijenta sadrži pojednostavljen prikaz dijagnoze i preporučene terapije, dok izvještaj za doktora obuhvata kompletnu historiju nalaza, prethodne dijagnoze i statistiku tretmana. Laboratorijski izvještaj sadrži samo rezultate analiza. Sva tri izvještaja koriste istu osnovnu strukturu, čime se postiže konzistentnost u sistemu, a istovremeno se zadržava fleksibilnost u prikazu specifičnog sadržaja.

Observer

Observer patern uspostavlja odnos jedan-prema-više između objekata tako da kada jedan objekat (subject) promijeni stanje, svi njegovi posmatrači (observeri) budu automatski obaviješteni i ažurirani. U sistemu HealthZone ovaj patern se prirodno preslikava na **notifikacioni mehanizam** – pacijenti, doktori i medicinske sestre moraju biti informisani o promjenama termina, novim nalazima ili statusu zahtjeva za opremu.

To možemo implementirati:

- definiranjem interfejsa Subject s metodama `dodajObserver()`, `ukloniObserver()` i `notifyAll()`;
- definiranjem interfejsa Observer s metodom `update()`;
- implementacijom konkretnih subjekata kao što su Termin, Nalaz i ZahtjevZaOpremu;
- registracijom Pacijenta, Doktora i Notifikacionog servisa kao observera.

Korištenje:

Kada doktor unese novi nalaz, pacijent (kao registrovani observer) automatski dobija notifikaciju putem e-maila ili push poruke u mobilnoj aplikaciji. Slično tome, otkazivanje termina od strane pacijenta automatski obavještava doktora i medicinsku sestru zaduženu za listu čekanja, koja zatim može ponuditi termin sljedećem pacijentu sa liste. Ovim pristupom postiže se labava sprega između izvora promjene i konzumenata informacije, što omogućava jednostavno dodavanje novih kanala obavještavanja u budućnosti.

Iterator

Iterator patern obezbjeđuje način za sekvencijalni pristup elementima kolekcije bez izlaganja njene interne strukture. U sistemu HealthZone postoji više kolekcija nad kojima je potrebno iterirati: lista nalaza pacijenta, lista termina doktora, lista čekanja, lista recenzija medicinske usluge i slično.

To možemo implementirati:

- definiranjem interfejsa `Iterator<T>` s metodama `hasNext()` i `next()`;
- kreiranjem konkretnih iteratora: `NalazIterator`, `TerminIterator`, `ListaCekanjaIterator`, `RecenzijaIterator`;
- implementacijom metode `getIterator()` u kolekcijama koje vraćaju odgovarajući iterator.

Korištenje:

Doktor može pregledati nalaze pacijenta hronološki, od najnovijeg ka najstarijem, koristeći `NalazIterator`, pri čemu mu interna struktura skladištenja (lista, mapa, baza) nije bitna. Medicinska sestra koristi `ListaCekanjaIterator` da prođe kroz pacijente po prioritetu i hitnosti, dok administrator može iterirati kroz sve recenzije date doktoru radi izračunavanja prosječne ocjene. Ovim se postiže ujednačen način pristupa kolekcijama bez obzira na njihovu konkretnu implementaciju.

Command

Command patern enkapsulira zahtjev kao objekat, čime omogućava parametrizaciju klijenata sa različitim zahtjevima, redove izvršavanja, logovanje i podršku za **undo/redo** operacije. U sistemu HealthZone administratorske i doktorske akcije često prolaze kroz više koraka i potrebna je mogućnost poništavanja, što je idealna primjena ovog paterna.

To možemo implementirati:

- definiranjem interfejsa Command s metodama execute() i undo();
- kreiranjem konkretnih komandi: ZakaziTerminCommand, OtkaziTerminCommand, KreirajNalazCommand, DodajKorisnikaCommand;
- klasa AdminController, DoktorController ili SestraController preuzima ulogu Invoker-a, dok servisi (TerminService, NalazService) djeluju kao Receiver.

Korištenje:

Kada administrator dodaje novog korisnika kroz više koraka (osnovni podaci, dodjela uloge, dodjela specijalizacije za doktore), svaki korak može biti enkapsuliran kao zasebna komanda. U slučaju greške ili promjene odluke, sistem može pozvati undo() i poništiti izmjene bez ručnog brisanja podataka. Isto vrijedi i za zakazivanje termina – ukoliko se naknadno otkrije da je termin pogrešno zakazan, komanda se može poništiti, čime se vraća prvobitno stanje slobodnih termina doktora.

Memento

Memento patern omogućava snimanje internog stanja objekta i njegovo kasnije vraćanje, bez narušavanja enkapsulacije tog objekta. U sistemu HealthZone ovaj patern je posebno koristan kod uređivanja medicinskih nalaza, gdje se zahtijeva mogućnost vraćanja na prethodne verzije dokumenta.

To možemo implementirati:

- definiranjem klase Nalaz kao Originator-a sa metodama spasiStanje() i vratiStanje();
- kreiranjem klase NalazMemento koja čuva trenutni snapshot nalaza (dijagnoza, terapija, datum izmjene);
- uvođenjem klase Caretaker (npr. IstorijaNalaza) koja čuva listu mementa za svaki nalaz.

Korištenje:

Doktor prilikom uređivanja nalaza svakim spašavanjem automatski kreira novi memento koji se pohranjuje u istoriju. Ukoliko naknadno utvrdi da je prethodna verzija dijagnoze bila preciznija, može jednostavno vratiti raniju verziju nalaza. Time se osigurava sljedivost i revizibilnost medicinske dokumentacije, što je posebno važno u kontekstu zdravstvenih standarda i pravne odgovornosti.

Mediator

Mediator patern centralizuje komunikaciju između većeg broja objekata, čime se smanjuje međusobna sprega i izbjegava se kompleksna mreža direktnih referenci. U sistemu HealthZone postoji više aktera (pacijent, doktor, medicinska sestra) koji moraju razmjenjivati informacije u realnom vremenu prilikom zakazivanja termina, izmjene rasporeda ili komunikacije oko nalaza.

To možemo implementirati:

- definiranjem interfejsa `KomunikacijaMediator` s metodama `posaljiPoruku()` i `registrujKorisnika()`;
- kreiranjem konkretne klase `HealthZoneMediator` koja koordinira komunikaciju između svih aktera;
- uvođenjem da svaki Korisnik šalje poruke isključivo kroz mediator, umjesto direktne reference na druge korisnike.

Korištenje:

Kada pacijent zatraži izmjenu termina, zahtjev se ne šalje direktno doktoru, već prolazi kroz `HealthZoneMediator` koji provjerava raspoloživost doktora, obavještava medicinsku sestru o promjeni u rasporedu i ažurira listu čekanja ukoliko je potrebno. Na ovaj način, klase `Pacijent`, `Doktor` i `MedicinskaSestra` ne moraju poznavati međusobne implementacije, što znatno olakšava održavanje i proširenje sistema novim ulogama ili kanalima komunikacije.

Zaključak

Uvođenjem paternna ponašanja u sistem **HealthZone** postiže se jasnija raspodjela odgovornosti između klasa, smanjuje se sprega između komponenti i povećava prilagodljivost sistema na buduće promjene. `Strategy` i `Template Method` omogućavaju fleksibilan prikaz podataka i generisanje izvještaja, `State` i `Command` pružaju kontrolu nad životnim ciklusom termina i mogućnost poništavanja akcija, dok `Observer` i `Mediator` olakšavaju komunikaciju između aktera u realnom vremenu. `Iterator` i `Memento` dodatno pojednostavljaju rad sa kolekcijama podataka i osiguravaju sljedivost medicinske dokumentacije.

Zajedno sa već implementiranim kreacijskim i strukturalnim paternima, paterni ponašanja zaokružuju arhitekturu sistema `HealthZone`, čineći ga skalabilnim, održivim i spremnim za buduća proširenja u skladu sa potrebama poliklinike.